IETF 127 · BOF PROPOSAL · ART AREA

MTA Hooks.

An HTTP protocol for delegating MTA processing decisions to external services.

Built on HTTP, JSON and CBOR, JSON Pointer and the JMAP message model.

draft-degennaro-mta-hooks-02
Mauro De Gennaro · Stalwart Labs LLC

The decision in front of this room

WE ARE ASKING

- Is this problem real?
- Is it well scoped?
- Is it worth solving here?
- Will people do the work?

WE ARE NOT ASKING

- Approve this wire format.
- Ratify a data model.
- Bless one implementation.

The draft exists as evidence that the problem is tractable, not as the thing on the table today.

Milter, in one slide

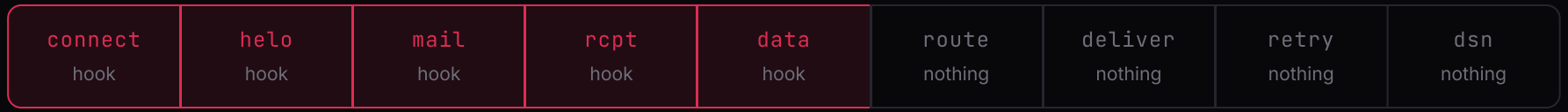
Out-of-process filtering protocol. Sendmail, around 2000. A C library, `libmilter`. Implemented by Sendmail and Postfix.



- Callbacks fire along the SMTP conversation
- The filter may reject, defer, discard, or modify the message
- Twenty five years of deployment, an enormous installed base
- Rspamd, ClamAV, SpamAssassin, OpenDKIM, OpenDMARC and the rest of the ecosystem all speak it
- It is the only portable way to filter mail today

This talk is not an argument that Milter is bad. It is an argument about reach.

Where Milter’s reach ends



The moment the server answers `250 Ok`, the filter is out of the picture for the rest of the message's life. It never learns whether what it accepted was delivered.

01

What this costs

Starting with a requirement that arrived this year and cannot be met at all.

DKIM2 asks for things that happen after acceptance

SECTION 8.6

The signature records the `RCPT TO` values used in the SMTP transaction that actually carried the message. If `bcc:` recipients are involved, each of them **MUST NOT** be revealed to any other recipient.

SECTIONS 12.1.4 AND 12.2

A DSN **MUST** carry its own signature, and **MUST** be addressed back to the MTA that sent the message, using the `mf=` tag of the highest numbered signature, not the return path.

SECTION 12.1.1.1

A forwarder receiving a DSN may propagate it to the address it was itself delivered under, after verifying the embedded message against the highest numbered Message-Instance field.

WHAT THAT ADDS UP TO

Signing per delivery transaction, not per queued message. A hook at DSN generation. And the ability to consume a DSN and originate a new one.

One Milter cannot produce that signature

WHEN THE FILTER RUNS

At `cleanup`. One queued message, one body, one recipient list. The delivery job has not been split yet.

WHEN THE SPLIT HAPPENS

Later, in `qmgr`, when recipients are grouped by transport and next hop. No filter runs there at all.

At signing time the filter has two options, and both are wrong:

- put every recipient in the signature, which discloses the blind copies to all of them
- guess, and sign a recipient set that does not match the transaction

The obstacle is not that the logic is spread across several filters. A filter cannot consolidate its way to data the server has not computed yet.

And then the delivery notifications

Generating one

There is no hook in the bounce path before the notification is injected. Nothing can attach a signature chain, and nothing can address it anywhere other than the return path.

The nearest approximation

Postfix can pass self-generated bounces through a filter after the fact. It acts globally on every bounce, runs after generation and never before, and the documentation warns that enabling content inspection of Postfix-generated mail is generally not safe.

Forwarding one

Recognise an inbound DSN, verify the embedded chain, correlate it to forwarding state held by this hop, and emit a new, re-signed notification addressed to the previous hop. A filter can add headers and recipients to the message in front of it. It cannot consume a message and cause the server to originate a different one.

Everything after acceptance, today

WHAT YOU NEED	HOW IT IS WIRED TODAY
Choose a transport or next hop	Lookup tables: one key string in, one value out
Transform content per destination	A filter loop with re-injection, giving the message a new identity midway
Archive with proof it was archived	BCC maps, with no return channel
Know whether recipient B received it	VERP addressing, a bounce mailbox, and log parsing
Report on delivery TLS	A dedicated side channel with its own collector chain
Suppress a notification	Global switches, or nothing

Not one of these is an IETF specification. Not one is portable between servers. Not one is correlated with the others, or with the decisions taken while the message was being received.

Context arrives by configuration, and silently does not

Macros

Every piece of context, the client address, the reverse name, TLS parameters, the authenticated identity, must be exported by the administrator, per stage, by name.

A forgotten macro is not an error. The field is simply absent, and the filter cannot tell "not configured" from "not available".

No negotiation

A filter cannot ask for what it needs, cannot discover what the server is able to provide, and cannot refuse to start when a required input is missing.

No authentication results

SPF, DKIM, DMARC and ARC are not part of the interface. Every filter either recomputes them or parses headers and establishes its own trust boundary.

Adding one new field

STEP 1

Patch the server core

Macros are a fixed vocabulary the core must know. There is no generic channel for extra data.

→

STEP 2

Define new macro names

Plus a documentation update, so implementers can discover that the field now exists.

STEP 3

Roll out the knowledge

Every filter author has to learn the new names and request them explicitly, or they never see the field.

→

RESULT

A core change per server

For one field. This is the real ceiling on what the interface can ever carry.

The strongest objection to Milter is not "can it carry X". It is "what does it cost to introduce X at all".

One message, one body

Milter modifies the message. There is no way to say that recipient A gets this body and recipients B and C get that one. Per-recipient content is met with a filter loop and re-injection.

WHAT THE LOOP COSTS

- Two passes through the queue for every message
- A new queue identity, so correlation needs log stitching
- One re-injection per recipient group
- Two policy points that must be kept consistent

WHO NEEDS IT

- Encryption gateways, per recipient
- Data loss prevention with a redacted variant
- Compliance footers that differ by destination
- DKIM2, for the reason two slides ago

Signals the interface cannot carry

CONNECTION AND SESSION	
Resets, no-ops, syntax errors	invisible
Session duration, command timing	invisible
Pregreet and early talkers	invisible
Pipelining violations	invisible
Forward-confirmed reverse DNS	filter resolves it itself

TRANSACTION AND TRANSPORT	
Client DSN options in use	invisible
Address rewriting, before and after	invisible
TLS fingerprint	absent or obsolete
Client certificate if unverified	never
Time budget for this call	never

And nothing carries provenance. A filter cannot learn that the previous filter quarantined the message, or why. The only channel between them is a header, which is unstructured, forgeable, and travels onward with the mail.

The people asking for this

Not the filter vendors

Rspamd, SpamAssassin and ClamAV paid the integration cost once and amortise it across every deployment. A written-down Milter specification changes little for them.

The operators

People running a server who want to add one site-specific rule. Check the recipient against our directory. Apply the footer policy legal asked for. Push an event into our SIEM. Ten to a hundred lines, in whatever language the rest of the shop already uses.

Stalwart ships Milter support. Its users asked for an HTTP interface anyway. That is a revealed preference from people who had both options in front of them, not a claim that binary protocols are hard.

Specifying Milter would not close the gap

It is a reasonable suggestion, and writing the format down would be useful. It would not address any of the previous slides.

REACH

A specification of the existing protocol still stops when the message is accepted. The empty half of slide four stays empty.

EXTENSIBILITY

Context still arrives by macro, and a new field still costs a core change in every server that implements it.

AUDIENCE

The operator with a fifty-line rule still cannot practically use it. That is the constituency asking for the work.

What is missing is not a document. Outbound delegation is not an unimplemented feature of Milter; there is nowhere in the protocol to put it. Its whole response vocabulary answers one question: what do I say to the client waiting on this command.

The problem, in short

Mail servers delegate decisions to external services. That delegation covers the conversation that receives a message, and stops there.

- Everything after acceptance is covered by a different mechanism in every server, and usually several unrelated ones at once
- None of those mechanisms is an IETF specification, portable, or correlated with the others
- The interface that does exist cannot carry a new field without a change to the server core
- Requirements now arriving from IETF work cannot be expressed through it at all
- The people who most need to write filters are the least able to

Is that the right problem, and is this the right place to solve it?

02

What we have built

Shown as evidence the problem is tractable, not as a proposal to ratify.

MTA Hooks

The server makes an HTTP request to a registered service at each processing stage, and applies what comes back.

WHY IT EXISTS

Operators asked for a way to write their own filters without linking against a C library. It grew from there.

WHAT IT COVERS

The whole life of a message: reception, routing, delivery, retry and notification, under one identity.

WHAT IT REUSES

HTTP, JSON and CBOR, JSON Pointer, and the JMAP message model. It specifies hook semantics and little else.

HTTP
TRANSPORT

9
STAGES

JSON / CBOR
SERIALISATION

Shipping
IN PRODUCTION

How it works

ONCE

Discovery

The service publishes a document at a well-known endpoint saying which stages it handles and which properties it understands.

→

ONCE

Registration

Server and service negotiate: these stages, this context, these modifications permitted, this failure behaviour.

PER MESSAGE

Hook call

An HTTP POST carrying exactly the context that was negotiated, and nothing else.

→

PER MESSAGE

Response

An action, and a set of patches describing what to change.

Negotiation is what makes the rest work. What is not advertised is known to be unavailable, so a service can adapt or refuse to start.

The hook request

A service registered only for `data` gets one call, with the context of every earlier stage already gathered. Skipping is real: the earlier stages produce no round trip for it.

```
{
  "stage": "data",
  "queue":     { "id": "01J8Z2K..." },
  "client":    { "ip": "203.0.113.10", "ptr": "mail.example.net", "fcrdns": "pass" },
  "tls":       { "version": "TLSv1.3", "cipher": "TLS_AES_256_GCM_SHA384" },
  "senderAuth":{ "spf": "pass", "dkim": "pass", "dmarc": "pass", "arc": "none" },
  "envelope":  { "from": "alice@example.org",
                 "to": [ { "address": "bob@example.com" } ] },
  "message":   { "subject": "Quarterly report", "bodyStructure": { } }
}
```

The parsed message, the raw bytes, or both. Authentication results come from the server, which already computed them.

The hook response

Patches against the request, so a service never echoes back a message it did not change.

```
{
  "set": [
    { "path": "/action",   "value": "reject" },
    { "path": "/response", "value": { "code": 550, "message": "Policy" } }
  ]
}
```

Inbound actions accept, reject, discard, quarantine, disconnect

Outbound actions continue, cancel, and per-recipient overrides

Modifications are declarative and bounded by what the registration permitted. An operator decides what a given service may change; the protocol does not assume it may change everything.

The stages



RECEIVING THE MESSAGE

- `route` fires before the next hop is resolved, per delivery job, and may route per recipient. The server splits the job accordingly
- `delivery` and `defer` report the real response from the real next hop, per recipient, per attempt, including the TLS outcome

DELIVERING IT, AND REPORTING ON IT

- `dsn` fires before a notification is sent, and may modify or suppress it
- Everything stays under one queue identity, so one message is one story rather than a set of records to be stitched together afterwards

Mapping back

THE PROBLEM	WHAT ANSWERS IT
Signature must match the delivery transaction	`route` fires after the job is split, per recipient, with per-recipient content
Notifications cannot be signed or redirected	`dsn` fires before the notification is sent, and may modify or suppress it
Nothing after acceptance	`route`, `delivery`, `defer` and `dsn` are stages in the same protocol
Context by macro, silently absent	Discovery and registration: a declared, negotiated contract
A new field costs a core patch	A reverse-domain extension namespace, negotiated bilaterally, ignored if unknown
One message, one body	Per-recipient variants at delivery time, without re-injection
No provenance between filters	The server orchestrates the chain and holds the authorship
Operators cannot write filters	An HTTP endpoint that accepts JSON and returns JSON

Status and open questions

Implementations

- **Stalwart**: complete, in production
- **Rspamd**: intent to implement stated
- **Haraka**: offered on the list
- Independent scanner implementations already on GitHub, written by people who are not the draft authors

Support without a commitment

Heinlein Group, Beonex, KTP Digital, the SAKE project

Open, and for a working group to decide

- How far routing delegation should go, and what bounds an operator sets
- Failure semantics: when to hold mail, when to let it pass
- Correlation across servers in a distributed deployment
- What belongs in the protocol and what belongs in deployment guidance

03

Over to the room

Is this a problem the IETF should take on, and are there enough of us to do the work?